PG4200: Algorithms and Data Structures
Prof. Dr. Rashmi Gupta
Kristiania University College, Oslo, Norway

Re-Sit Exam

- This exam contains six major questions, some with multiple parts. You have 24 hours to earn 100 points.
- This exam is an open book. You may use handwritten A4 sheets, MS Word documents for theoretical answers, and IntelliJ IDE for Java code implementation. The use of calculators or programmable devices is permitted.
- Please ensure your handwriting is **legible** (I cannot grade that I cannot read!). Before you upload your solutions, ensure all your answers are included and matched numerically with your question paper.
- Start uploading your exam paper/file ahead of time, as it may take a long time to upload.
- Exam papers that are not handed in on Wiseflow by the specified time of the submission date will not proceed to assessment. No late solutions will be accepted.
- Before submitting, remember to check that all files can be opened and that every file is included. It may be a good idea to check the saved files on several machines before submitting them in Wiseflow.
- Incorrect file format or lacking documents may result in the submission not being passed or assessed.
- Do not spend too much time on any one Question. Read them through first, and start solving them in the order that allows you to make the most progress.
- Show your work, as partial credit will be given. You will be graded not only on the correctness of your answer but also on the clarity with which you express it.
- The use of ChatGPT is strictly prohibited. Your submitted documents will be checked for plagiarism.

Good luck!

Questions	Title	Type of Explanation	Points	Score
1	LO1: Understanding Data Structures	Algorithmic	15	
2	LO1: Understanding Data Structures	Algorithmic/code implementation	15	
3	LO2: Searching Algorithms	Algorithmic/graphical	15	
4	LO 3: Sorting Algorithms	Algorithmic/graphical	15	
5	LO 4: Traversing Graphs Algorithms	Algorithmic/code/graphical	20	
6	LO 5 and LO 6: Computability and Complexity	short answers	20	
Total			**100**	

Question 1 (15 points, Formula and Calculation): LO1: Understanding Data Structures

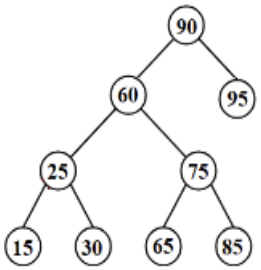
- How to access any random element from a 1-D array? Where the given base address of an array **A[1300 …… 1900]** is **1022**, and the size of each element is **2 bytes** in the memory, find the address of **A[1704].**

Question 2 (15 points, Code and Explanation): LO1: Stack (push/pop/getMin operations)

- You are given **N** elements , and your task is to Implement a Stack in which you can get a minimum element in O(1) time. You are required to complete the three methods:
  - **push()** which takes one argument, an integer **'x'** to be pushed into the stack,
  - **pop()** which returns an integer popped out from the stack, and
  - **getMin()** which returns the min element from the stack. (-1 will be returned if for **pop() and getMin()** the stack is empty.)
  - **Constraints:**
    1 <= Number of queries <= 100
    1 <= values of the stack <= 100

Question 3 (15 points, Algorithms and step-by-step graphical solution): LO2: Searching Algorithms

- Delete **node 60** in this Binary Search Tree. Provide step-by-step algorithm and human representation way of solving the problem.



Question 4 (15 points, Algorithm and Graphical Representation): LO 3: Sorting Algorithms

- Sort the following list with **Merge Sort** and provide an algorithm and a graphical representation of solving the given problem:

Array = *{70, 50, 30, 10, 20, 40, 60}*

Question 5 (20 points, Code): LO 4: Traversing Graphs Algorithms

- Regardless of how fast computers become or how cheap memory gets, efficiency will always remain an important consideration. To show the importance of developing efficient algorithms, compare these two algorithms:
  - Recursive *algorithm to solve fib*(*n*) for $1 \leq n \leq 5$ Fibonacci term
  - Iterative *algorithm to solve fib*(*n*) for $1 \leq n \leq 5$ Fibonacci term

  Justify which algorithm is more efficient and why through the solution with code.

Question 6 (20 points, Short Answer): LO 5 and LO 6: Computability and Complexity

1. What is Complexity Class? Why is it known as complexity classes in data structures?
2. Why do we use Big O Notation to analyse algorithms' complexities in data structures?
3. What is P class, and What are the features of this class? Provide a real-life example of this type of problem.
4. What is NP class, and What are the features of this class? Provide a real-life example of this type of problem.
5. What is an NP-complete class, and What are the features of this class? Provide a real-life example of this type of problem.